

PARTIE 1 — SCRIPT DES SLIDES

SLIDE 1

Titre / Introduction

Bonjour, je suis Kenza FOUDALI, stagiaire au sein de CF Consult à Casablanca. Durant ce stage, j'ai eu l'opportunité de concevoir et développer une application mobile Android pour la gestion de l'inventaire physique de l'entreprise. C'est ce projet que je vais vous présenter aujourd'hui.

SLIDE 2

Présentation de l'organisme d'accueil

CF Consult est un cabinet de conseil et d'expertise comptable basé à Casablanca. L'entreprise accompagne ses clients dans la gestion administrative, financière et organisationnelle. Elle doit tenir à jour un inventaire précis de son patrimoine matériel — ordinateurs, mobilier, matériel de bureau — et c'est justement là qu'est née l'idée de ce projet.

Inventaire Mobile CFC — Application Android

Contexte et besoin

Avant ce projet, le suivi de l'inventaire se faisait manuellement, sur papier ou via des fichiers Excel. Cette méthode posait plusieurs limites : risques d'erreurs de saisie, pas de traçabilité en temps réel, et impossibilité d'accéder aux données depuis le terrain. L'objectif était de moderniser ce processus en passant à une solution mobile, rapide et fiable. Il ne s'agissait pas d'un problème bloquant, mais d'une vraie opportunité de digitalisation.

Kenza FOUDALI — Stagiaire CF Consult Casablanca

Script complet | Questions du jury | Abréviations & Glossaire

Objectifs du projet

Trois objectifs principaux : (1) Centraliser toutes les données d'inventaire dans une base de données structurée. (2) Offrir une interface mobile intuitive pour saisir, modifier et consulter les équipements depuis le terrain. (3) Intégrer des fonctionnalités avancées : scan de codes-barres et reconnaissance IA.

Planification — Diagramme de Gantt

Le projet s'est déroulé sur environ deux mois : analyse des besoins et cahier des charges, puis conception des diagrammes UML (cas d'utilisation, classes, séquences), développement du backend Spring Boot, puis de l'application Android, et enfin deux semaines de tests et corrections. Cette planification m'a permis de livrer un produit fonctionnel dans les délais.

SLIDE 6

Architecture globale — 3 tiers

J'ai choisi une architecture trois tiers, standard pour les applications client-serveur :

- Tier 1 — Présentation : l'application Android (Java natif) — affichage et interactions.
- Tier 2 — Métier : le backend Spring Boot — logique applicative, sécurité JWT, API REST.
- Tier 3 — Données : base de données PostgreSQL — stockage persistant.

Cette séparation permet de réutiliser le même backend pour une future application web sans le modifier.

SLIDE 7

Backend Spring Boot

Le backend suit les couches classiques de Spring Boot :

Controllers REST → reçoivent les requêtes HTTP, retournent du JSON.

Services → logique métier (validation, règles de gestion).

Repositories → interface avec PostgreSQL via Spring Data JPA.

Entités → classes Java mappées sur les tables BD avec annotations Hibernate.

La sécurité : Spring Security + JWT. Le token encode rôle et identité, vérifié par un filtre à chaque requête sans interroger la BD (stateless).

SLIDE 8

Application Android

Développée en Java natif — contrôle total sur les performances et APIs Android. Architecture MVC : Activities (contrôleurs), Layouts XML (vues), Modèles (entités).

Communication backend via Retrofit2 qui déserialise automatiquement le JSON via GSON. Session utilisateur en SharedPreferences : token JWT, rôle, nom sauvegardés localement.

SLIDE 9**Authentification JWT — flux complet**

1. Utilisateur saisit login/mot de passe → POST JSON vers /auth/login.
2. Backend vérifie dans PostgreSQL (mdp haché BCrypt).
3. Si OK → génère JWT signé (clé secrète) → renvoyé à l'app.
4. App stocke le token en SharedPreferences.
5. Chaque requête suivante : header Authorization: Bearer .
6. Filtre Spring Security decode et vérifie le token avant chaque contrôleur.

SLIDE 10**Scan de codes-barres — ZXing**

ZXing (Zebra Crossing) = référence open source pour codes-barres et QR codes sur Android. Intégration via IntentIntegrator : on lance l'Intent vers l'Activity ZXing, elle ouvre la caméra, détecte le code, renvoie le résultat via onActivityResult. Le numéro de série est ensuite utilisé pour rechercher l'équipement via Retrofit.

SLIDE 11**Reconnaissance IA — Google Gemini 2.5 Flash**

Fonctionnalité la plus avancée. Flux complet :

1. Agent prend une photo → image encodée en Base64.
2. Requête HTTP POST vers l'API Gemini (sans passer par notre backend).
3. Prompt : répondre uniquement en JSON {designation, type, marque, etat, description}.
4. App parse le JSON → pré-remplit le formulaire automatiquement.
5. Agent vérifie et valide. Quelques secondes vs plusieurs minutes manuellement.

SLIDE 12**Fonctionnalités principales**

- Équipements informatiques : créer, consulter, rechercher, modifier, supprimer (désignation, marque, N° série, état, localisation, responsable).
- Autres actifs (mobilier) : module séparé adapté.

- Export Excel / PDF de l'inventaire complet.
- Gestion des droits : utilisateur (créer/consulter) vs administrateur (supprimer/admin).

SLIDE 13

Base de données PostgreSQL

SGBD relationnel robuste et open source. Tables principales :

- utilisateurs (login, password BCrypt, rôle, nom, prénom)
- équipements (toutes caractéristiques matérielles)
- autres_actifs (mobilier)

Relations via clés étrangères. Hibernate génère le SQL automatiquement depuis les annotations Java. ddl-auto=update : schéma mis à jour au démarrage sans SQL manuel.

SLIDE 14

Technologies utilisées

Backend : Java 17, Spring Boot 3, Spring Security, Spring Data JPA / Hibernate, PostgreSQL, Maven.

Mobile : Android Java, Retrofit2, OkHttp, ZXing, Material Design 3.

Outils : Android Studio, IntelliJ IDEA, Postman.

SLIDE 15

Démonstration — Résultats

Application entièrement fonctionnelle : connexion → navigation → scan code-barres (retrouve l'équipement instantanément) → photo IA (identification automatique) → validation → enregistrement. Quelques secondes au lieu de plusieurs minutes, sans risque d'erreur de saisie.

SLIDE 16

Conclusion & Perspectives d'évolution

Solution full-stack complète : de la BD jusqu'à l'interface mobile, avec API REST sécurisée JWT, IA Gemini, scan ZXing.

Perspectives d'évolution :

1. Notifications push — Firebase Cloud Messaging pour alerter les responsables.
2. Mode hors-ligne — Room Database (SQLite) + synchronisation différée.
3. Migration Kotlin — syntaxe concise et coroutines pour l'asynchrone.
4. Tests unitaires — JUnit (backend) et Espresso (UI Android).

PARTIE 2 — QUESTIONS DU JURY & RÉPONSES

ARCHITECTURE

Q : Pourquoi une architecture 3 tiers plutôt que 2 tiers ?

Une architecture 2 tiers — Android connecté directement à PostgreSQL — exposerait les credentials de la BD dans l'APK, extractibles par décompilation. Avec 3 tiers, seul le backend accède à la BD. La logique métier est centralisée : une modification backend s'applique à tous les clients sans re-déployer l'app.

Q : Pourquoi Spring Boot pour le backend ?

Auto-configuration (peu de boilerplate), Spring Security complet pour l'authentification, Spring Data JPA pour l'ORM, Tomcat embarqué sans serveur externe. Standard industriel très utilisé en entreprise.

Q : Pourquoi PostgreSQL plutôt que MySQL ou SQLite ?

SQLite = fichier local, non adapté au multi-utilisateurs. PostgreSQL gère mieux les transactions concurrentes que MySQL — important quand plusieurs agents modifient l'inventaire simultanément. Meilleure conformité aux standards SQL.

SECURITE & AUTHENTIFICATION

Q : Expliquez précisément comment fonctionne JWT.

JWT (JSON Web Token) = 3 parties en Base64 séparées par des points : (1) Header : algorithme de signature (HS256). (2) Payload : claims — userId, role, date d'expiration. (3) Signature : calculée avec une clé secrète côté serveur. À chaque requête, le backend recalcule la signature et compare. C'est stateless : pas de session stockée côté serveur.

Q : Comment les mots de passe sont-ils stockés ?

Jamais en clair. BCrypt = algorithme de hachage adaptatif avec salt aléatoire par mot de passe. Deux utilisateurs avec le même mot de passe auront des hashes différents. Le facteur de coût peut être augmenté pour résister au brute force. Spring Security gère via BCryptPasswordEncoder.

Q : Que se passe-t-il si le token JWT expire ?

Erreur 401 Unauthorized → l'app redirige vers l'écran de connexion. Amélioration possible : refresh token (longue durée) permettant de renouveler le token d'accès sans redemander les credentials.

IA GEMINI

Q : Comment fonctionne l'intégration de Gemini techniquement ?

Image lue via `ContentResolver.openInputStream()` → encodée en Base64. JSON construit avec tableau 'contents' > 'parts' : prompt texte + image inline (type MIME + data Base64). POST via `OkHttp` vers l'API Gemini. Réponse JSON parsée → extraction du texte du premier candidat → parsé à son tour en JSON pour récupérer désignation, type, marque, état, description.

Q : Pourquoi Gemini 2.5 Flash et pas ChatGPT ?

Multimodal natif (conçu images + texte dès le départ), très rapide (suffixe Flash), quota gratuit suffisant pour une démo, et s'intègre via HTTP standard sans SDK obligatoire.

Q : Quels risques liés à la clé API dans l'app Android ?

L'APK peut être décompilé, exposant la clé. En production : faire passer les appels Gemini par notre backend qui détient la clé en variable d'environnement, et exposer un endpoint `/ai/analyze`. La clé n'est jamais côté client.

ANDROID & TECHNIQUE

Q : Pourquoi Java Android natif et pas Flutter ou React Native ?

Accès direct à toutes les APIs Android sans couche d'abstraction — plus fiable pour caméra, `FileProvider`, permissions. C'est aussi le langage maîtrisé en cours.

Q : Comment fonctionne le scan de codes-barres techniquement ?

`ZXing` analyse les frames vidéo en continu, détecte les patterns (barres verticales pour code 1D, carrés pour QR). L'algorithme Reed-Solomon corrige les erreurs de lecture et extrait la chaîne encodée. `IntentIntegrator` délègue tout ce traitement à l'Activity `ZXing`.

Q : Différence entre Retrofit et OkHttp dans votre code ?

`OkHttp` = couche HTTP bas niveau (connexions TCP, timeouts, intercepteurs). `Retrofit` = abstraction au-dessus transformant des interfaces Java annotées en appels HTTP. J'utilise `Retrofit` pour mon backend (plus propre et maintenable), `OkHttp` direct pour Gemini car le JSON multimodal doit être construit manuellement.

Q : Comment gérez-vous les erreurs réseau ?

Callbacks Retrofit : `onFailure` (réseau/timeout) et `onResponse` (HTTP reçu). Je vérifie `response.isSuccessful()` pour distinguer succès et erreurs HTTP, et affiche des Toasts appropriés. En mode démo (admin/admin), l'app fonctionne hors ligne.

Q : Perspectives d'évolution ?

1. Notifications push via Firebase Cloud Messaging pour alerter les responsables.
2. Mode hors-ligne complet avec Room Database et synchronisation différée.
3. Migration vers Kotlin — syntaxe concise et coroutines pour l'asynchrone.
4. Tests unitaires avec JUnit (backend) et Espresso (UI Android).

PARTIE 3 — ABREVIATIONS & GLOSSAIRE

Terme	Définition
JWT	JSON Web Token — jeton d'authentification signé encodé en Base64, stateless, sécurise les API REST.
API	Application Programming Interface — interface de communication entre logiciels via requêtes HTTP JSON.
REST	Representational State Transfer — style d'architecture pour API web basé sur HTTP (GET, POST, PUT, DELETE).
HTTP/HTTPS	Protocole de communication web. HTTPS = version chiffrée via SSL/TLS.
JSON	JavaScript Object Notation — format léger d'échange de données textuelles : {"cle":"valeur"}.
MVC	Modèle-Vue-Contrôleur — pattern séparant logique (Modèle), affichage (Vue) et contrôle (Contrôleur).
ORM	Object-Relational Mapping — fait correspondre classes Java et tables SQL automatiquement (ex: Hibernate).
JPA	Java Persistence API — spécification Java pour la persistance relationnelle. Impl: Spring Data JPA/Hibernate.
APK	Android Package Kit — fichier d'installation Android, peut être décompilé (d'où le risque clé API).
SDK	Software Development Kit — ensemble d'outils pour développer sur une plateforme donnée.
UI	User Interface — interface utilisateur, ce que l'utilisateur voit et avec quoi il interagit.
UML	Unified Modeling Language — langage de modélisation (diagrammes classes, séquences, cas d'utilisation...).
SGBD	Système de Gestion de Bases de Données — logiciel gérant stockage et accès aux données (PostgreSQL...).
SQL	Structured Query Language — langage de requête pour bases de données relationnelles.
BCrypt	Algorithme de hachage de mots de passe avec salt aléatoire, résistant aux attaques brute force.
Base64	Encodage transformant des données binaires (image) en texte transportable en JSON.
OkHttp	Bibliothèque HTTP bas niveau pour Android/Java — gère connexions TCP, timeouts. Utilisée par Retrofit.
Retrofit2	Bibliothèque Android transformant des interfaces Java annotées en appels HTTP (couche au-dessus d'OkHttp).

GSON	Bibliothèque Google convertissant automatiquement objets Java ↔ JSON.
ZXing	Zebra Crossing — bibliothèque open source de lecture codes-barres et QR codes sur Android.
Room	Bibliothèque Android (SQLite) pour persistance locale avec synchronisation possible.
FCM	Firebase Cloud Messaging — service Google d'envoi de notifications push vers mobiles.
JUnit	Framework de tests unitaires Java — vérifie le bon fonctionnement des méthodes backend.
Espresso	Framework de tests UI Android — simule les interactions de l'utilisateur sur l'interface.
Gemini	Modèle d'IA multimodal de Google (texte + image). 2.5 Flash = version rapide, gratuite, via API HTTP.
Spring Boot	Framework Java pour créer des applications web et API REST avec configuration automatique.
Hibernate	Implémentation de JPA — génère automatiquement les requêtes SQL depuis les annotations Java.
Maven	Outil de gestion des dépendances et de build pour projets Java (fichier pom.xml).
Postman	Outil graphique pour tester manuellement les endpoints d'une API REST.
Material Design 3	Système de design Google pour Android — composants UI modernes (boutons, cartes, champs...).
FileProvider	Mécanisme Android sécurisé pour partager des fichiers entre applications (ex: photo → caméra).
SharedPreferences	Stockage clé-valeur persistant et léger sur Android — utilisé pour la session utilisateur.
Intent	Objet Android représentant une action (ouvrir une Activity, lancer la caméra...).
Activity	Composant Android représentant un écran de l'application avec son cycle de vie.
3 tiers	Architecture logicielle : Présentation (Android) → Métier (Spring Boot) → Données (PostgreSQL).
Stateless	Sans état : le serveur ne stocke pas de session. JWT est stateless — token vérifié à chaque requête.
Salt (BCrypt)	Valeur aléatoire ajoutée au mot de passe avant hachage, rendant chaque hash unique.
Reed-Solomon	Algorithme de correction d'erreurs utilisé par ZXing pour décoder les codes-barres même partiellement lisibles.